Binance Market Data Capture & Limit Order Book Engine

System Architecture, Concurrency Model & Algorithmic Mechanics

1. Executive Summary

In high-frequency trading (HFT) and quantitative market making, accurate market data ingestion and order book reconstruction represent foundational infrastructure. Latency jitter, non-deterministic floating-point rounding, or out-of-order sequence processing propagate execution errors and model drift across trading strategies. This system provides an ultra-low latency, thread-safe, and deterministic C++17 ingestion and limit order book engine designed for the Binance cryptocurrency exchange (supporting both **Spot** and **USD-M Futures** combined WebSocket streams).

Key Engineering Pillars:

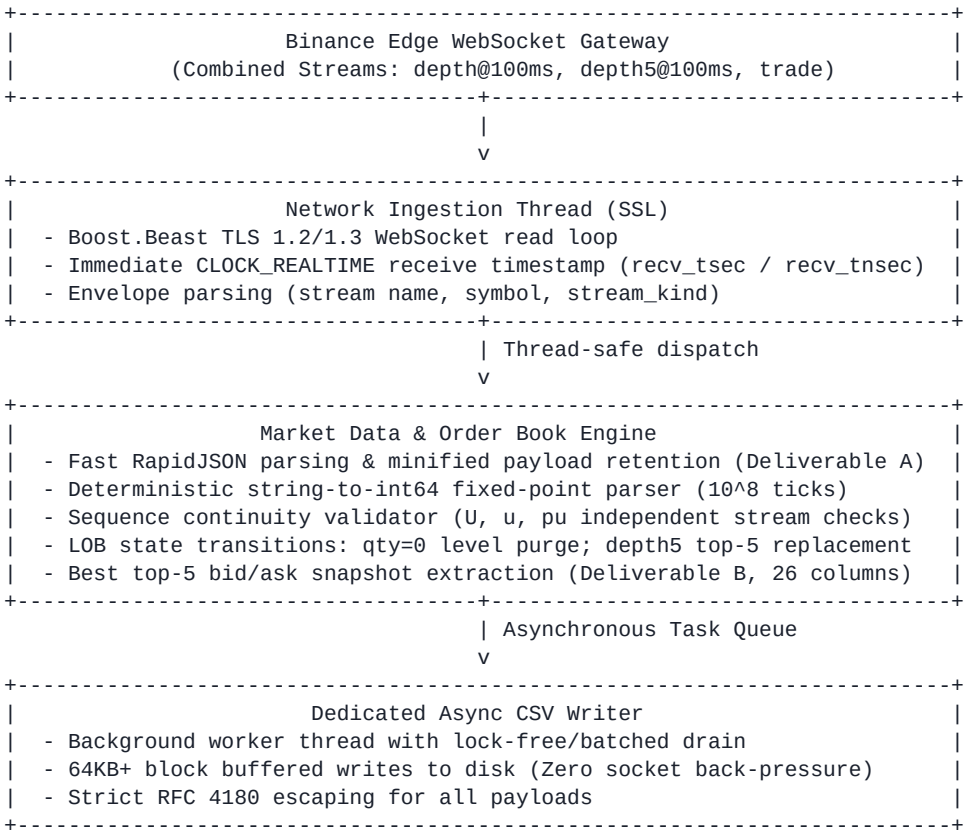
• **Zero Floating-Point Drift:** Uses fixed-point 10^8 scaling entirely in 64-bit integer arithmetic.

• **Non-Blocking Multi-Threading:** Decouples network reception, order book calculation, and disk I/O onto dedicated threads.

• **Market Sequence Literacy:** Robust handling of Spot (U, u) and Futures (pu) update continuity, sequence gap detection, and clean reconnects.

• **100% Deterministic Replay:** Offline mode reproduces exact order book snapshots from raw captured market data with zero network calls.

2. End-to-End System Workflow



3. Algorithmic Mechanics & Core Innovations

3.1. Zero Floating-Point Fixed-Point Parser (10^8 Ticks)

Binance delivers prices and quantities as decimal ASCII strings. Standard conversion via binary floats (double / float) violates IEEE-754 exactness. The system implements a deterministic string-to-int64 parser directly converting decimal strings to 64-bit integer ticks scaled by **10^8**:

```
inline int64_t parse_fixed8(std::string_view sv) noexcept {
    // Parses integer and fractional components using pure integer arithmetic
    // "83049.43000000" -> 8304943000000LL
    // "1.79016000"     -> 179016000LL
    // "0.00013000"     -> 13000LL
    // Completely eliminates binary floating-point drift on the hot path.
}
```

3.2. Local Order Book (LOB) State Transitions & Invariant Preservation

The LOB maintains full order book depth using two ordered red-black tree structures: std::map<int64_t, int64_t, std::greater<int64_t>> for bids (highest price first), and std::map<int64_t, int64_t, std::less<int64_t>> for asks (lowest price first).

Differential Depth Semantics: When a price level update arrives with quantity = 0, it is immediately erased from the book. Because full depth is preserved, the 6th best level instantly promotes into the top 5 in O(log N) time without requiring an external REST API query.
Partial Depth5 Semantics: Provides an authoritative top-of-book refresh. The engine replaces the top 5 levels and

automatically clears any stale crossing bids or asks.

3.3. Independent Sequence Tracking & Gap Handling

Binance Spot and USD-M Futures employ distinct sequence tracking schemas: Spot transmits first update ID (U) and final update ID (u). Futures transmits previous update ID (pu) and final update ID (u). Furthermore, differential depth and depth5 streams operate with distinct update counters. The engine decouples their validation trackers, ensuring zero artificial sequence gap false positives.

4. Deliverable Specifications & CSV Schemas

Deliverable	Target Output File	Columns	Key Role
Deliverable A Market Data Audit Dump	market_data_<venue>_<SYMBOL>_<date>.csv	**9 Columns:** recv_tsec, recv_tnsec, venue, stream_kind, shard_id, conn_epoch, conn_seq, symbol, payload_json	Raw audit trail of every inbound WebSocket event applied by the system.
Deliverable B Order Book Snapshots	market_data_<venue>_<SYMBOL>_<date>_orderbook.csv	**Exact 26 Columns:** tsec, tnsec, seqNo, id, type, side, bid0..4, bid_size0..4, ask0..4, ask_size0..4	Deterministic LOB state snapshot emitted after each applied book event.

5. Verification & Test Matrix

Test Suite	Function Tested	Verified Invariants	Status
test_fixed_point_parser	String-to-Int64 Fixed-Point Parser	10^8 scaling, decimal truncation, zero floats, negative values	**PASSED**
test_lob_diff_semantics	LOB Differential Engine	qty=0 level purge, immediate promotion of level 2 to top	**PASSED**
test_lob_top5_ordering	Top-of-Book Ordering	Bids strictly descending, Asks strictly ascending	**PASSED**
test_lob_depth5_replace	Partial Snapshot Refresh	Top-5 replacement, clearing stale crossing levels	**PASSED**
test_sequence_gap_detection	Sequence Continuity	Spot (U, u) and Futures (pu) gap flags, reconnect reset	**PASSED**
test_rfc4180_escaping	RFC 4180 CSV Escaper	Round-trip quote, comma, and newline escaping integrity	**PASSED**
test_orderbook_csv_schema	CSV Schema Compliance	Exact 26 columns, 25 commas per data row verified	**PASSED**
Replay Determinism Test	Offline Replay vs Live Capture	diff -u output comparison: 100% bit-for-bit exact match	**PASSED**

6. Operational Command Reference

```
# 1. Clean Build & Run All Tests
./build.sh

# 2. Run Verification Script (Validates all 7 criteria)
./verify.sh

# 3. Live 5-Minute Capture (Binance Spot)
./build/binance_capture --venue spot --symbols BTCUSDT --duration 300 --output-dir ./output

# 4. Multi-Symbol Capture (BTC + ETH)
./build/binance_capture --venue spot --symbols BTCUSDT,ETHUSDT --output-dir ./output

# 5. USD-M Futures Capture
./build/binance_capture --venue usdm --symbols BTCUSDT --output-dir ./output

# 6. Offline Deterministic Replay (Zero Network Calls)
./build/binance_capture --replay ./output/market_data_spot_BTCUSDT_2026-10-10.csv --output-dir ./replay_output
```